

Especificação de Requisitos de Software

Documento 6 de 8 — Arquitetura Tecnológica

Postinder — Plataforma de Gestão e Aprovação de Conteúdo

Agosto de 2026 · Versão 1.0

Sumário

1 Visão geral	1
2 Stack tecnológico	2
2.1 Frontend (apps/admin)	2
2.2 Backend (backend)	2
2.3 Banco de dados e infraestrutura	2
3 Estilo arquitetural	3
3.1 Frontend	3
3.2 Backend	3
4 Modelo de autenticação e autorização	3
5 Arquitetura de dados e Storage	3
6 Arquitetura de integrações externas	4
7 Topologia de implantação de referência	5
8 Configuração e variáveis de ambiente	6
9 Build, empacotamento e reprodutibilidade	6
10 Integração e entrega contínua (estado atual)	6
11 Componentes do sistema	7

1 Visão geral

O Postinder é organizado como duas aplicações independentes que se comunicam por API REST/JSON sobre HTTPS: um **frontend administrativo** em React (apps/admin), que também hospeda o Portal do Cliente, e um **backend** em Express/TypeScript (backend), organizado por módulos de domínio. A persistência é feita em PostgreSQL, e os arquivos de mídia são armazenados em disco local (desenvolvimento) ou no Supabase Storage (produção).

apps/admin	React + Vite (Área Administrativa + Portal do Cliente)
backend	Express + TypeScript (API REST, módulos de domínio)
database	migrations versionadas (PostgreSQL)
docs	guias de arquitetura e operação

2 Stack tecnológico

2.1 Frontend (apps/admin)

Camada	Tecnologia
Framework de UI	React 18
Bundler / dev server	Vite 5
Roteamento	React Router DOM 6
Estado global	Zustand 5
Cache/estado de servidor	TanStack Query 5
Cliente HTTP	Axios
Formulários e validação	React Hook Form + Zod (resolvers @hookform/resolvers)
Estilização	Tailwind CSS
Ícones	lucide-react
Gráficos	Recharts
Notificações (toast)	react-hot-toast
Testes unitários	node --test (utilitários)
Testes de componentes React	Vitest + Testing Library (jsdom)
Empacotamento de tipos	TypeScript 5

2.2 Backend (backend)

Camada	Tecnologia
Runtime	Node.js 24.x
Framework HTTP	Express 4
Linguagem	TypeScript 5 (execução via tsx)
Banco de dados	PostgreSQL, acesso via driver pg
Autenticação	jsonwebtoken (JWT), bcryptjs (hash de senha)
Upload de arquivos	multer
Processamento de imagem (branding)	sharp, pngjs
Validação de esquema	zod
Identificadores	uuid
Storage em produção	@supabase/supabase-js (Supabase Storage)
Testes	node --test (nativo do Node.js)

2.3 Banco de dados e infraestrutura

Componente	Detalhe
SGBD	PostgreSQL (Docker local em desenvolvimento; Supabase em produção)
Controle de esquema	Migrations SQL versionadas, registradas em schema_migrations
Storage de arquivos	Diretório uploads/ local (desenvolvimento) ou Supabase Storage (produção), por adaptador comum
Hospedagem de referência	Vercel (frontend) · Render (API) · Supabase (Postgres + Storage)

3 Estilo arquitetural

3.1 Frontend

Aplicação single-page (SPA) organizada por **features** (`src/features/{auth,clients,posts,approvals,dash`), com componentes compartilhados em `src/components` (layout, mídia, notificações, busca, branding, IA), camada de serviços HTTP em `src/services`, estado global em `src/store` (Zustand) e utilitários em `src/utils`. O Portal do Cliente reutiliza os mesmos componentes de mídia e formatação da área administrativa, evitando duplicação de lógica de exibição de imagens, vídeos e legendas.

3.2 Backend

Organização **modular por domínio** (`backend/src/modules/<módulo>`), inspirada em arquitetura em camadas: os módulos mais complexos (`auth, posts, clients, soundtracks, branding, integrations`) separam explicitamente `domain` (regras de negócio puras), `application` (serviços e DTOs de orquestração), `infrastructure` (repositórios de acesso a dados e adaptadores externos) e `presentation` (controllers e rotas HTTP). Módulos mais simples (`users, activities, notifications, approvals, portal, maintenance`) mantêm uma estrutura mais direta de `infrastructure/presentation`. Elementos transversais — autenticação, autorização por capacidade, banco de dados, tratamento de exceções, middlewares, upload e utilitários — residem em `backend/src/shared`.

A composição das rotas ocorre em `backend/src/app.ts`, que:

1. Configura CORS restrito a origens permitidas (`APP_PUBLIC_URL, CORS_ORIGINS, localhost:5173`).
2. Expõe rotas públicas de saúde (`/health, /health/db, /health/storage`) e de autenticação (`/api/v1/auth`).
3. Expõe o portal por token (`/api/v1/portal/:token`) e a leitura pública de branding (`/api/v1/branding`), sem exigir autenticação administrativa.
4. Aplica a cadeia `authMiddleware` → `clientAuthMiddleware` ao portal autenticado (`/api/v1/client-portal`).
5. Aplica a cadeia `authMiddleware` → `adminAuthMiddleware` → `requireAdminRouteCapability` a todo o bloco de rotas administrativas (`/api/v1/{notifications,posts,clients,users,approvals,fi`).
6. Monta a rota de manutenção de demonstração **condicionalmente**, apenas quando o ambiente a habilita explicitamente.

4 Modelo de autenticação e autorização

O sistema mantém **quatro contextos de token distintos e não intercambiáveis**: JWT administrativo (`type: "admin"`), JWT de Cliente (`type: "client"`), refresh tokens (com contexto próprio) e token privado de portal (hash + cópia cifrada recuperável). A autorização administrativa é **baseada em capacidades** (`AdminCapability`), resolvida por rota em `AdminRoutePolicy` e aplicada centralmente pelo middleware `requireAdminRouteCapability` — qualquer rota administrativa sem política declarada nega acesso por padrão (HTTP 403). Os quatro papéis oficiais (`admin, manager, editor, viewer`) recebem conjuntos de capacidades distintos, conforme detalhado na matriz do Documento 2.

5 Arquitetura de dados e Storage

A identidade de cada objeto de arquivo é o par **bucket + storage_path**, independente da URL pública (mantida apenas para exibição/compatibilidade). Um adaptador comum abstrai o Storage local (desenvolvimento) e o Supabase Storage (produção), permitindo que o restante do sistema opere de forma agnóstica ao ambiente. Uploads são recebidos em memória pelo multer

e enviados sequencialmente ao Storage; falhas de persistência em banco disparam compensação (remoção) do objeto já enviado.

6 Arquitetura de integrações externas

Integração	Direção	Situação
Anthropic API (IA — geração de insights)	Server-side, exclusiva do backend	Habilitada, sob capacidade <code>ai-insights:generate</code> , <code>allowlist</code> de modelo e <code>timeout</code>
Z-API (WhatsApp)	Server-side (planejada)	Estrutura prevista; não habilitada nas rotas atuais
Twilio	Server-side (planejada)	Desabilitada até implementação segura
GoHighLevel	Server-side (planejada)	Desabilitada até implementação segura
Canva	Server-side (planejada)	Desabilitada até implementação segura
Resend (e-mail transacional)	Server-side (planejada)	Desabilitada até implementação segura
Google Analytics	Client-side	Apenas identificador público (<code>VITE_GA_MEASUREMENT_ID</code>)

Nenhuma credencial de integração é exposta ao frontend; o fluxo de IA segue estritamente Frontend → API Postinder → autenticação/contexto → autorização por capacidade → controller → service → adaptador Anthropic (server-side).

7 Topologia de implantação de referência

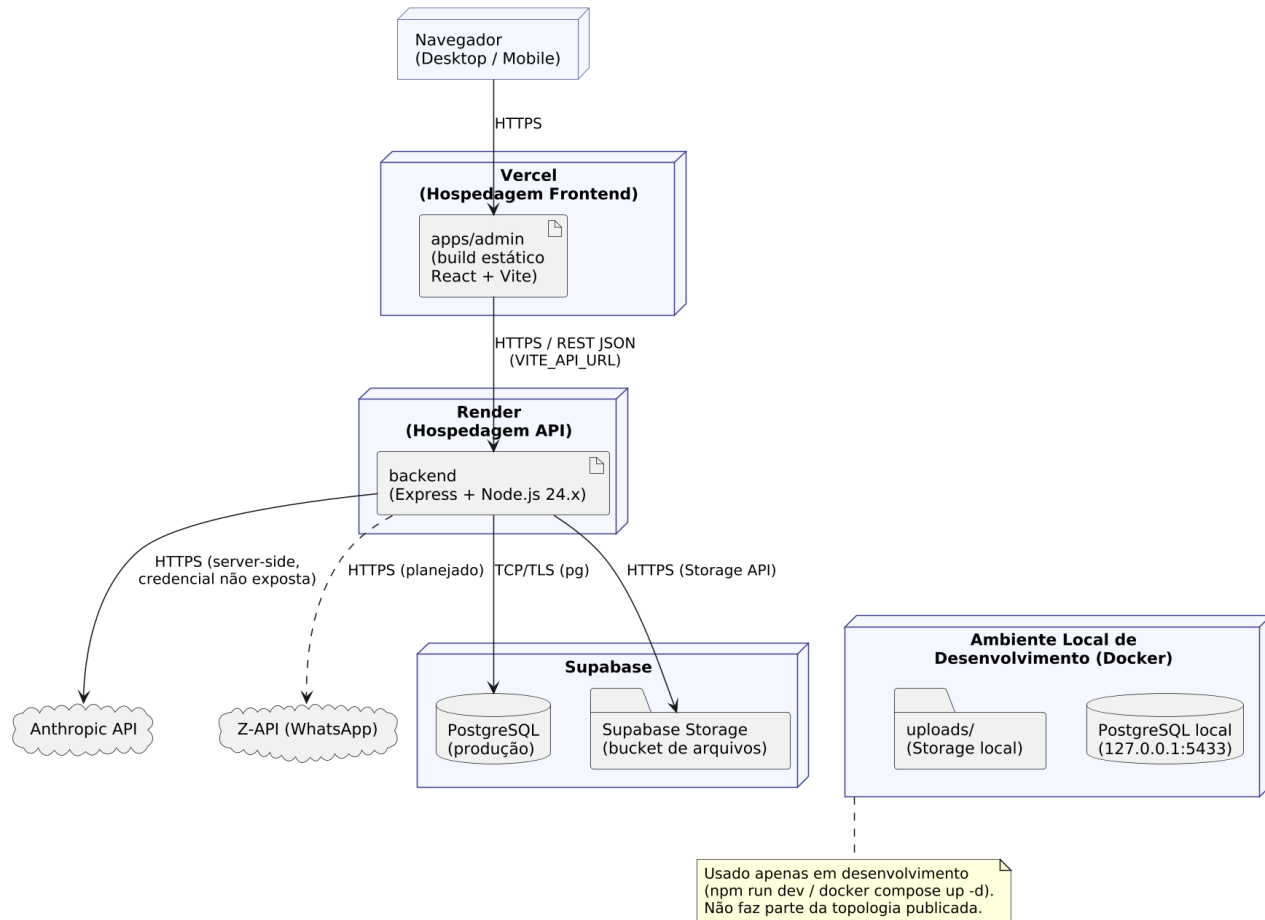


Figura 1: Diagrama de Implantação

Camada	Provedor de referência	Observações
Frontend (build estático)	Vercel	Consome apenas variáveis VITE_* públicas
API (backend)	Render	npm run db:migrate como <i>release step</i> bloqueante antes do <i>Start Command</i>
Banco de dados	Supabase (PostgreSQL)	Migrations como única fonte de verdade estrutural
Storage de arquivos	Supabase Storage	Bucket público no fluxo atual; URLs assinadas não implementadas
Desenvolvimento local	Docker Compose (PostgreSQL) + diretório uploads/	Fora da topologia publicada

8 Configuração e variáveis de ambiente

O sistema distingue três variáveis de propósito distinto para evitar ambiguidade entre modo técnico e finalidade de implantação:

- **NODE_ENV** — modo técnico do processo Node.js (development, production, etc.).
- **DEPLOYMENT_MODE** — finalidade da implantação (production ou demo); controla, por exemplo, a disponibilidade do reset de demonstração.
- **VITE_DEPLOYMENT_MODE** — apenas apresentação no frontend; não é capaz de habilitar comportamentos sensíveis no backend.

Variáveis públicas consumíveis pelo frontend: `VITE_API_URL`, `VITE_DEPLOYMENT_MODE`, `VITE_GA_MEASUREMENT_ID`. Nenhuma outra variável com prefixo `VITE_*` deve conter segredos.

9 Build, empacotamento e reprodutibilidade

- Node.js 24.x (fixado por `.node-version` em 24.16.0) e npm 11.13.0 em todos os artefatos implantáveis.
- Lockfiles versionados no formato v3 na raiz, em backend e em apps/admin; uso de `npm ci` em desenvolvimento, validação e implantação.
- `.npmrc` próprio por artefato, aplicando `engine-strict=true` mesmo quando o gerenciador de pacotes é iniciado diretamente no subdiretório.
- Build do backend via `tsc` (compilação TypeScript → dist/); execução em produção via `node dist/server.js`.
- Build do frontend via `tsc && vite build`.

10 Integração e entrega contínua (estado atual)

Não há, na versão avaliada, um fluxo de integração contínua (CI) configurado no repositório. O script `npm run lint` existe no frontend, mas a configuração do ESLint necessária para executá-lo ainda não está disponível; consequentemente, o lint não participa das configurações versionadas ou documentadas de Vercel e Render. A validação de qualidade concentra-se atualmente na suíte de testes automatizados (backend e frontend) executada manualmente antes de cada publicação, complementada por um checklist de deploy documentado (`docs/deploy/CHECKLIST_DEPLOY_TESTE.md`).

11 Componentes do sistema

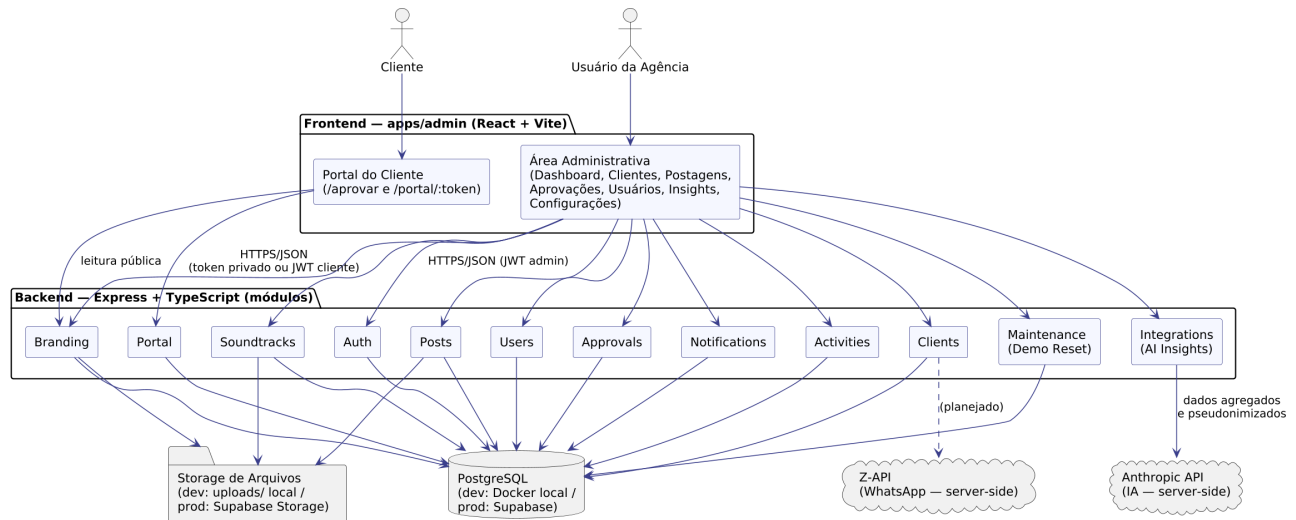


Figura 2: Diagrama de Componentes